

VIDEO NOTES · PART 3

Setting Up an Industry-Grade Project

Structure, utilities and a provider-agnostic LLM client

What this document is

A plain-English walkthrough of the first hands-on session —
how a real LangChain project is organised before any features are built.

1 Why Structure Comes First

This session is deliberately not about clever AI features. It is about the groundwork that makes everything after it manageable.

Most LangChain tutorials live inside a single notebook. That is fine for learning one idea, but it falls apart the moment a project grows: prompts get pasted in three places, there is no way to see what happened when something fails, and switching model provider means editing code all over the file.

One notebook

Everything in one file
Prompts copy-pasted around
No logs when things break
Provider hard-coded in place

Fine for a demo

A structured project

One folder per concern
Prompts stored in one place
A logger recording every run
Provider set in a config file

Ready for real work

Figure 1 — The difference is not the AI. It is whether the project survives growth.

The mindset behind this lecture: build the scaffolding once, properly, then drop each new LangChain component into a slot that already exists.

2 The Setup Sequence

The session follows a fixed order. Each step depends on the one before it.

1. Virtual
environment



2. Install
dependencies



3. Generate
folders



4. Build utilities



5. Test

Figure 2 — The five steps of the session, in order.

Step 1 — A dedicated virtual environment

A virtual environment is a private, isolated space for this project's packages. Nothing it installs leaks into your system, and nothing installed elsewhere interferes with it.

Without it, two projects needing different versions of the same library will eventually break each other — and the failure is usually confusing when it happens.

Step 2 — Installing from a requirements file

Rather than installing packages one at a time, every dependency is listed in a single requirements file and installed together.

Why this matters: that file is the recipe for the project. Anyone who clones the repository — a teammate, an interviewer, or you on a new machine — gets an identical setup from one command.

3 Generating the Project Structure

Instead of creating folders by hand, the session uses a small script called **template.py**. Running it once creates the entire directory layout, with empty placeholder files where code will go.

template.py
run once



A complete, consistent folder layout
every directory and placeholder file created for you

Figure 3 — One script produces the whole skeleton, the same way every time.

The layout it produces

```
project/
├─ src/
│   ├─ llms/ ← model provider clients
│   ├─ prompts/ ← prompt templates
│   ├─ memory/ ← chat history handling
│   ├─ output_parsers/ ← turning replies into data
│   └─ utils/ ← logger, file loader, helpers
├─ config/ ← config.json lives here
├─ logs/ ← written automatically at run time
├─ requirements.txt
├─ template.py
└─ test_app.py
```

Notice that the folder names match the LangChain components from the previous lecture. That is intentional — when you learn chains or memory next, you already know exactly where the code belongs.

A habit worth copying: folders named after concepts, not after file types. A folder called *prompts* tells a reader what the project does. A folder called *scripts* tells them nothing.

4 The Utility Modules

Three small helper files are built next. None of them touch AI at all — they exist to make everything else easier to run and debug.

Logger

records what happened

File Loader

reads JSON and text

Common Helpers

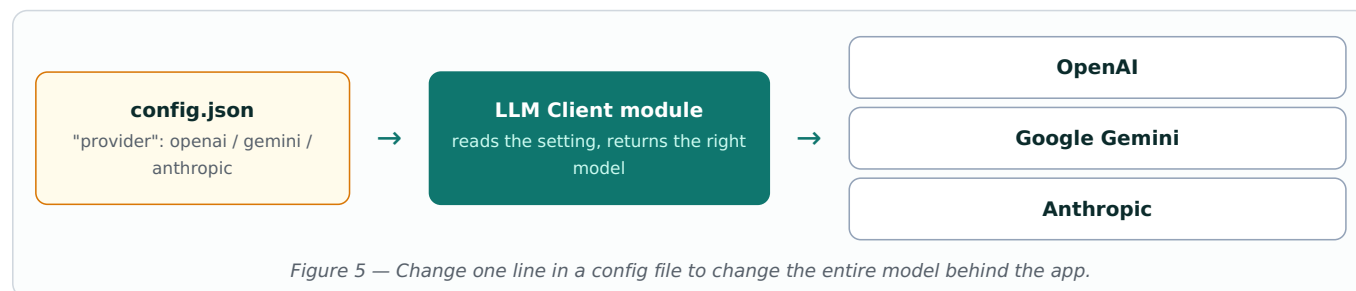
separators, timing

Figure 4 — The three utilities that support every other part of the project.

Utility	What it does and why you want it
Custom logger	Writes a record of each run to a log file instead of scattering print statements through the code. When a prompt misbehaves in a week's time, the log is how you find out what was actually sent and what came back.
File loader	One place that knows how to read JSON and text files. Configuration, prompt files and sample data all load through it, so file handling is not rewritten in five places.
Common helpers	Small conveniences: printed separators that make terminal output readable, and an execution timer that shows how long a call took — useful, since LLM calls are the slow and costly part of any run.

5 The LLM Client and Config File

This is the centrepiece of the session. All model access goes through a single module, and which provider it uses is decided by a configuration file — not by the code.



The rest of the application never asks which provider is in use. It simply requests a model from this module and gets one back. That single decision is what makes the whole project portable.

The pattern behind it: this is the LangChain idea from the previous lecture — a common interface across providers — applied at the project level. LangChain standardises how models are called; this module standardises how *your* code chooses one.

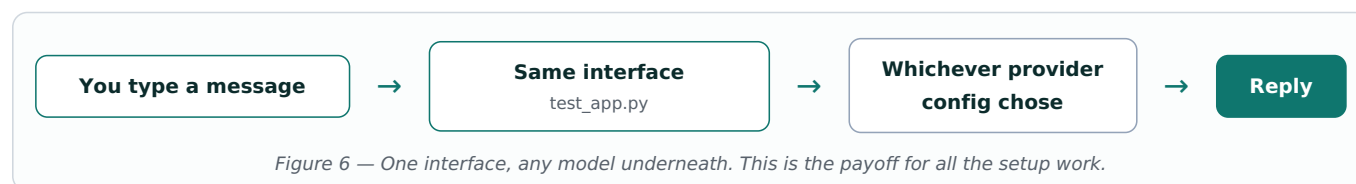
Why keep the setting outside the code

- **No code edits to switch.** Trying a different provider becomes a config change, not a code change — so it cannot introduce a bug.
- **Different settings per environment.** A cheap fast model while developing, a stronger one in production, from the same codebase.
- **Secrets stay out of source control.** API keys belong in environment variables and config, never committed alongside your code.

6 Proving It Works

The session finishes with **test_app.py** — a small interactive chat script in the terminal.

Its purpose is not to be impressive. It is to prove one specific claim: the same chat interface works unchanged no matter which provider the config file names.



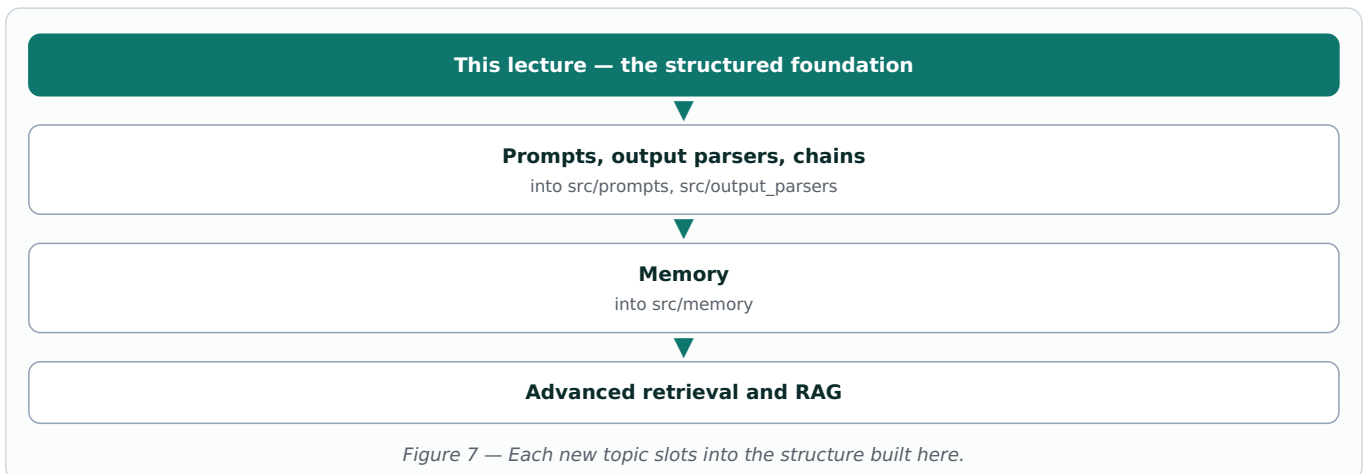
7 What You Have at the End

Piece	Its job in the project
Virtual environment	Keeps this project's packages isolated from everything else
requirements.txt	Reproduces the exact setup on any machine
template.py	Creates the full folder skeleton in one run
Logger	Records each run so failures can be traced later
File loader	Single, consistent way to read JSON and text
Common helpers	Readable output and execution timing
LLM client module	The one place that knows how to reach a model
config.json	Chooses the provider without touching code
test_app.py	Confirms the whole setup actually works end to end

Worth noticing: not one line of this is specific to a particular AI feature. That is exactly the point — the foundation is finished, so the next lectures can be purely about LangChain concepts.

8 The Road Ahead

With the scaffolding in place, upcoming lectures go component by component — each one landing in a folder that already exists.



Before the next lecture

- ☐ Create the virtual environment and confirm it activates.
- ☐ Install everything from the requirements file in one go.
- ☐ Run the template script and look through the folders it made.
- ☐ Trigger the logger once and find the log file it wrote.
- ☐ Switch the provider in config.json and rerun the test app.
- ☐ Keep your API keys out of the repository — check before committing.

The short version: a virtual environment for isolation, a generated folder structure for organisation, small utilities for visibility, and a config-driven client so the model provider is a setting rather than a decision baked into your code.